第9-10课时｜多智能体协作与编排框架

MBA课程：大模型智能体

90分钟　讲授 + 演示 + 实操

- 主题：从"单兵作战"到"组织化智能"
- 重点：必要性、协作模式、通信方式、框架选型、落地挑战
- 定义：多智能体（Multi-Agent）系统 = 多个角色化智能体在统一目标与规则下协同完成复杂任务

你将获得什么

1. 判断：什么时候应该上多智能体系统，什么时候不该
2. 设计：掌握四种主流协作模式
3. 工程：理解通信协议、状态管理、观测与治理
4. 选型：会比较主流框架与产品生态（含 Semantic Kernel / Dify / Coze）
5. 即时体验：5分钟上手 Cursor / Claude Code / OpenClaw

课程地图

1. 为什么需要多Agent
2. 四种协作模式
3. Agent间通信方式
4. 2026主流产品与框架生态对比
5. 企业落地挑战与治理
6. 5分钟即时体验与作业

Part A｜为什么需要多Agent?

从能力边界到组织能力

- 单Agent擅长"短链任务"
- 多Agent擅长"复杂系统任务"
- 关键不是模型更大，而是分工更优

单Agent的典型瓶颈

- **上下文瓶颈**：信息越多越混乱
- **角色冲突**：既要"创造"又要"审查"
- **长链推理脆弱**：中间步骤容易漂移
- **不可并行**：所有环节串行执行
- **质量无保险**：缺少独立校验角色

复杂度阶梯：任务越复杂，越需要"组织"

任务类型	例子	单Agent可行性	多Agent价值
简单问答	概念解释	高	低
单步生成	一段文案	高	低
多步分析	行业研究	中	中高
决策建议	投资/战略建议	低中	高
持续运营	客服、风控、供应链	低	很高

把多智能体系统看成"公司组织"

- CEO：目标设定与资源分配（Supervisor）
- 经理：拆解任务、跟进进度（Orchestrator）
- 专家：执行专业子任务（Worker Agents）
- 审计：质量与风险把关（Critic/QA Agent）

> 智能体系统的本质：**角色 + 流程 + 规则**

Anthropic: 多Agent研究系统

> 来源: [How We Built Our Multi-Agent Research System](#) (Anthropic, 2025)

架构：Lead Agent + 并行Subagents

```
→ Lead Agent (    +    )
      ├→ Subagent1:        A
      ├→ Subagent2:        B
      └→ Subagent3:        C
  ←     → Citation Agent →
```

效果：多Agent系统比单Agent提升**90.2%** (BrowseComp评测)

为什么多Agent有效?

Token使用解释80%的性能差异

因素	性能贡献
Token用量	80%
工具调用次数	~10%
模型选择	~5%

核心洞察：多Agent本质是"花足够多的token解决问题"

权衡：Agent用4×token，多Agent用15×token (vs 普通对话)

Anthropic的Prompting原则

1. **像Agent一样思考** — 用Console模拟，观察失败模式
2. **教会Orchestrator如何委派** — 详细任务描述，避免重复/遗漏
3. **努力匹配查询复杂度** — 简单任务1个Agent，复杂任务10+个
4. **工具设计至关重要** — 差的工具描述会误导Agent
5. **让Agent自我改进** — 用Claude优化Prompt和工具描述
6. **先宽后窄** — 从宽泛搜索开始，逐步聚焦
7. **引导思考过程** — 用Extended Thinking做可控的草稿纸

多智能体系统的商业价值（ROI视角）

- 提升覆盖度：多视角减少漏项
- 提升可靠性：交叉验证减少幻觉
- 提升效率：并行执行缩短端到端时间
- 提升复用性：角色组件可复用到不同业务

注意：ROI成立前提是任务复杂度足够高

反例：什么时候不该用多Agent

- 需求单一、一次性、无复杂依赖
- 输出容错高，不需要严格验证
- 实时性要求极高（子秒级）
- 预算非常有限

原则：能用单Agent稳定解决，就先别上多Agent。

案例快照：电商增长周报

旧流程（人工）

- 数据拉取 → 分析 → 写作 → 审核 = 1.5天

多Agent流程

- Data Agent + Analyst + Writer + QA Agent 并行/串行混合
- 总耗时降到 2~3 小时，且结构更标准化

5分钟即时体验｜本课Lab安排

- **体验A（5分钟）**：访问 cursor.com 体验 Composer
- **体验B（5分钟）**：观看/跟做 Claude Code 终端演示
- **体验C（5分钟）**：进行一轮 OpenClaw 多Agent对话

记录三项观察：

1. 完成速度
2. 多文件/多角色协作控制感
3. 结果可追踪性

本段小结：必要性判断框架

1. 看复杂度：是否存在多专业子任务
2. 看风险：是否需要质量把关
3. 看时效：是否可从并行中获益
4. 看成本：token、延迟、工程维护是否可接受

Part B｜四种协作模式

模式决定系统上限

- 不是"哪个框架好"，而是"模式是否匹配业务"

四种协作模式总览

模式	核心结构	典型优点	典型风险
流水线（Pipeline）	顺序交接	可控、清晰	累积误差
主管-执行（Supervisor）	中心调度	灵活、可中断	调度压力集中
辩论（Debate）	多角色博弈	决策质量高	成本高、轮次长
群体/黑板（Swarm/Blackboard）	去中心化协作	可扩展、鲁棒	一致性难

模式1：流水线 Pipeline

结构：`Research -> Analysis -> Draft -> Review`

适用：

- 输出形态稳定（报告、SOP、周报）
- 过程可标准化

关键设计：

- 每一步输出模板化
- 强制输入/输出契约（Schema）

Pipeline 架构图（文本）

```
User Request
    |
    v
[Research Agent] -> facts.json
    |
    v
[Analyst Agent]  -> insights.json
    |
    v
[Writer Agent]   -> draft.md
    |
    v
[QA Agent]       -> final.md
```

Pipeline 的KPI与治理点

- KPI：成功率、平均耗时、重试率、人工介入率
- 治理点：
  - 每步"可回放日志"
  - 每步"质量闸门"
  - 失败"只回滚局部，不全链路重跑"

动手试试 A｜访问 Cursor Composer（5分钟）

- 打开：cursor.com
- 创建一个包含 2~3 个文件的小项目（如 `README.md` + `plan.md` + `todo.md`）
- 在 Composer 输入：
  - "请基于这3个文件，生成一版可执行的一周迭代计划，并直接改写文件。"

观察点：

- 是否能跨文件理解上下文
- 修改是否可预览、可回退
- 你是否仍然保持"主导权"

模式2：Supervisor（主管调度）

- 一个"主管Agent"动态决策下一步调用谁
- 可以根据中间结果改变路径

适用：

- 需求经常变化
- 分支路径较多
- 需要 Human-in-the-loop

Supervisor 流程示意

```
            +-----------------+
            |  Supervisor Agent |
            +---------+---------+
                      |
      +---------------+---------------+
      |                               |
      v                               v
[Research Agent]                 [Data Agent]
      |                               |
      +---------------+---------------+
                      |
                      v
                [Critic Agent]
                      |
                      v
                [Writer Agent]
```

Supervisor 的决策策略

- Rule-based：基于规则路由（最可控）
- Score-based：基于评分路由（更智能）
- Learned Router：训练路由器（最强但最重）

建议：MBA业务先从 Rule-based 起步。

动手试试 B｜Claude Code终端演示（5分钟）

- **官网**：Claude Code
- 准备一个本地项目目录
- 在终端发出自然语言指令（示例）：
  - "扫描本目录，找出可优化的3处代码并分别提交修改建议。"
- 重点观察 Claude Code 的：
  - 多文件连续编辑能力
  - 命令行协作反馈（diff、确认、回滚）
  - 与你的人机分工边界

模式3：辩论 Debate

- 至少两个立场Agent + 一个裁判/整合Agent
- 通过"反驳-修正"提升决策鲁棒性

适用：

- 战略决策
- 风险评估
- 投资建议

Debate 典型回合

1. 正方提出方案与证据
2. 反方指出漏洞与反例
3. 双方补充数据
4. 裁判汇总"共识与分歧"
5. 输出"可执行建议 + 风险清单"

Debate 的防跑偏机制

- 每轮限制最大字数和证据数量
- 强制引用来源（URL/数据日期）
- 设定停止条件（轮次上限、收敛阈值）
- 裁判必须输出"不确定性说明"

动手试试 C｜OpenClaw多Agent对话（5分钟）

- 用同一个业务问题发起一轮多Agent协作（例如"AI客服上线评审"）
- 建议至少包含：CEO/Manager/Coder/Verifier 等角色
- 对比单Agent结果，记录：
  - 任务拆解是否更清晰
  - 争议点是否被显式暴露
  - 最终结论是否更可审计

模式4：Swarm / Blackboard

- 多个Agent向共享"黑板"写入中间成果
- 其他Agent读取后继续补全

适用：

- 大规模探索
- 创新方案生成
- 异步协作

Blackboard 数据结构

```
{
  "task_id": "go-to-market-2026",
  "hypotheses": [],
  "evidence": [],
  "risks": [],
  "open_questions": [],
  "owner": "swarm"
}
```

Swarm 的关键问题

- 如何避免重复劳动?
- 如何做冲突合并?
- 如何评估"收敛"?

常见解法:

- 去重索引 + 相似度聚类
- 冲突投票 + 裁决Agent
- 目标函数阈值（coverage/confidence）

课堂观察卡 1｜协作行为记录

- 哪个工具/产品在"任务拆解"阶段最清晰?
- 哪个在"执行速度"上最突出?
- 哪个在"过程可追踪"上最好?

> 用 3 句话写下你的小组观察结论。

模式选择矩阵（实战版）

业务特征	推荐模式
固定产出、规范流程	Pipeline
路径多变、需要调度	Supervisor
高风险决策	Debate
大规模探索/创新	Swarm

混合模式：企业常态

- 前段用 Supervisor 分解任务
- 中段关键节点用 Debate 复核
- 后段用 Pipeline 规范产出
- 跨团队问题用 Blackboard 汇总

> 真实系统通常是"组合拳"，不是单模式。

本段小结：模式先于框架

- 先定义协作模式，再选技术框架
- 模式不清，框架再强也会乱
- 设计时要先画"职责图"和"消息图"

Part C｜通信方式与状态管理

多Agent系统的"神经系统"

通信的三要素

1. **消息格式**：发送什么
2. **传输机制**：怎么送达
3. **状态语义**：如何继续下一步

四种通信方式

方式	特点	适用场景
点对点（Direct）	简单直接	小规模协作
消息总线（Bus）	松耦合	多团队扩展
黑板（Blackboard）	共享记忆	异步协作
事件驱动（Event）	响应快	实时流程

通信协议建议（最小可行版）

```
{
  "message_id": "uuid",
  "from": "research_agent",
  "to": "analyst_agent",
  "task_id": "task_2026_001",
  "type": "EVIDENCE",
  "payload": {"facts": []},
  "confidence": 0.82,
  "timestamp": "2026-02-27T00:20:00Z"
}
```

消息类型设计

- `PLAN`：计划与拆解
- `EVIDENCE`：证据与数据
- `CRITIQUE`：质疑与漏洞
- `DECISION`：决策与结论
- `ALERT`：异常与风险

统一类型 = 可追踪、可审计、可自动化。

状态管理：短期记忆 vs 长期记忆

- 短期记忆：当前任务上下文（会话级）
- 长期记忆：知识库、策略库、历史结果（系统级）

实践建议：

- 短期存流程状态
- 长期存"可复用资产"

上下文压缩策略

- 摘要压缩：保留结论+证据索引
- 结构压缩：把文本转为JSON字段
- 分层上下文：核心信息常驻、细节按需加载

避免：把所有历史对话无差别塞进Prompt。

冲突解决机制

冲突类型	机制
事实冲突	以来源可信度加权
观点冲突	辩论 + 裁判投票
任务冲突	Supervisor优先级调度
资源冲突	限流 + 排队

一致性与收敛

- 设定收敛标准：
  - 关键结论一致率 > 80%
  - 风险项覆盖率 > 90%
  - 不确定项清单完整
- 达不到收敛标准时：升级人工复核

可观测性（Observability）

核心指标：

- Latency（端到端时延）
- Cost（token/工具成本）
- Quality（评分/通过率）
- Stability（重试率/超时率）

课堂观察卡 2｜指标打分（2分钟）

给你刚才体验的工具各打 1~5 分：

- 上手难度
- 协作效率
- 结果质量
- 可治理性（日志/审批/回滚）

输出：一张小组评分表。

本段小结：通信决定可维护性

- Schema先行，日志可回放
- 状态显式化，避免"黑箱串话"
- 没有观测，就没有优化

Part D｜2026主流产品与框架生态

先看产品，再看框架，再做治理

2026产品全景图（课堂版）

类别	代表产品	核心价值
通用对话与多模态入口	ChatGPT（GPT-4o）	通用任务覆盖广，适合作为企业AI统一入口
终端编码Agent	Claude Code / Codex CLI	在真实代码库中完成端到端协作
AI编程IDE	Cursor	人机共编 + 多文件上下文编辑
实时信息Agent	Grok	连接实时社交语境与事件流
多智能体编排系统	OpenClaw	CEO架构，专业智能体分工协作

Claude Code｜产品定位（2026）

- Anthropic 面向工程团队的编码Agent
- 主战场：**终端 + 代码仓**，强调"在你现有工作流内协作"
- 强项：理解项目结构、跨文件改动、连续执行任务

适合：

- 快速修复 bug
- 重构多个相关模块
- 先改后验（修改 + 测试 +解释）

Claude Code｜典型终端协作流

1. 用自然语言给任务目标（如"修复登录超时并补测试"）
2. Agent 扫描仓库，提出改动计划
3. 分批修改多个文件并展示 diff
4. 人类确认后执行测试/格式化/提交建议

管理者关注：

- 是否可回退
- 是否可审计
- 是否可配置权限边界

Claude Code｜优势与边界

优势

- 终端友好，接入成本低
- 多文件协作效率高
- 与工程实践（测试、lint）结合自然

边界

- 需要团队具备基础工程规范
- 高风险改动仍需人工review把关

OpenClaw｜CEO架构多Agent系统

- 核心思想：把复杂任务组织化
- 典型结构：CEO → Manager → Specialist Agents
- 支持 push-based 协作与子任务并行

适合：

- 跨职能复杂任务（产品 + 研发 + 验证）
- 需要明确职责边界的团队流程

OpenClaw｜7个专业Agent协作

示例角色（课程版本）：

1. CEO（目标与优先级）
2. Manager（任务拆解与调度）
3. Researcher（信息收集）
4. Coder（实现）
5. Verifier（质量审查）
6. Deployer（交付上线）
7. Reporter（总结与同步）

> 价值：把"隐性脑内流程"变成"显性组织流程"。

OpenClaw｜课堂业务流示例

业务题目：上线"AI客服质检"流程

- CEO：定义质量与成本目标
- Manager：分解为数据、规则、验证三个子任务
- Specialists：并行推进
- Verifier：做风险审查与回归验证
- Reporter：输出决策日志与结论

结果：更清晰的责任链与可追踪决策链。

Cursor｜AI编程IDE定位

- IDE内原生AI协作体验
- 强项：项目级上下文理解 + 即时编辑建议
- 适合"边看边改"的开发节奏

常见入口：

- Chat（问答）
- Composer（多文件协作编辑）
- Inline Edit（局部改写）

Cursor｜Composer多文件协作

Composer 能做什么：

- 一次任务触达多个文件
- 自动维护文件间一致性（接口、类型、引用）
- 先给计划，再执行改动

课堂建议：

- 用小项目先体验"多文件改写 + 回退确认"
- 重点观察你对改动的控制感

Cursor｜团队导入建议

- 先从低风险模块试点（文档、测试、脚手架）
- 建立"AI改动必须可解释"的review规则
- 形成统一提示词模板与验收清单

KPI建议：

- 需求到首版代码时间
- review返工率
- 缺陷泄漏率

Grok｜xAI实时Agent定位

- 特点：实时信息敏感度高
- 与 X 平台语境连接紧密
- 擅长"事件驱动"的观点整合与追踪

适合：

- 舆情监控
- 品牌危机快速研判
- 热点事件的策略草案

Grok｜X平台集成场景

场景示例：新品发布日舆情监测

1. 追踪关键话题与情绪波动
2. 抽取争议点与传播节点
3. 输出响应建议（澄清/客服/公关）

注意：

- 实时信息 ≠ 事实真相
- 高风险决策必须二次核验

Codex CLI｜OpenAI命令行Agent

- 在终端中完成"理解-修改-执行"的编码闭环
- 优势：可脚本化、可融入CI/CD前置流程
- 适合偏自动化的工程团队

典型用途：

- 批量重构
- 自动补测试
- 迁移脚本生成

Codex CLI｜典型命令工作流

1. 指定任务目标与约束（测试必须通过）
2. 读取仓库并生成改动计划
3. 执行多文件修改并产出说明
4. 运行测试并回报结果

落地关键：

- 权限控制
- 命令白名单
- 审计日志留存

框架生态对比（新增版）

框架	核心范式	典型优势	典型场景
AutoGen	对话协作	多角色讨论自然	辩论/评审
CrewAI	角色+任务	上手快、业务友好	流程化执行
LangGraph	状态图编排	可控、可恢复、可审计	企业级流程
Semantic Kernel	插件与规划器	微软生态集成强	企业应用集成
Dify	可视化工作流	低门槛构建Agent应用	业务快速落地
Coze	Bot平台/工作流	平台化发布与运营便捷	运营与增长场景

免责说明：本表用于课堂教学的相对比较，不构成采购建议；各框架能力会随版本快速变化，请以官方最新文档与实测为准。

六框架横向对比（治理维度）

维度	AutoGen	CrewAI	LangGraph	Semantic Kernel	Dify	Coze
学习曲线	中	低中	中高	中高	低	低
开发速度	快	快	中	中	快	快
流程控制	中	中	高	中高	中	中
可观测性	中	中	高	中高	中	中
企业治理	中	中高	高	高	中	中
生态集成	中	中	高	高	中	中高

免责说明：评分基于课堂通用场景（中等复杂度、需要一定治理）的经验判断，不代表对任一产品的绝对优劣结论。

选型建议（2026课堂版）

1. **要快上线**：先 Dify / Coze / CrewAI 做业务验证
2. **要强治理**：优先 LangGraph / Semantic Kernel
3. **要多轮决策讨论**：补充 AutoGen 能力
4. **要工程效率**：在研发侧引入 Claude Code / Cursor / Codex CLI
5. **要跨职能协同**：用 OpenClaw 组织多Agent分工

本段小结：产品层 + 框架层 + 治理层

- 2026年选型不再是"单一框架之争"
- 产品决定体验上限，框架决定工程下限
- 真正可落地的系统 = 能跑 + 可控 + 可审计

Part E｜企业落地挑战

多Agent不是Demo，而是系统工程

挑战1：成本爆炸

成本来源：

- 多Agent多轮对话
- 工具调用频繁
- 失败重试链路过长

控制手段：

- 动态裁剪上下文
- 低价值环节降级模型
- 设置预算上限与熔断

挑战2：时延与死锁

- 多Agent互等导致停滞
- 外部工具超时拖慢全链路

工程措施：

- 每步超时 + 回退策略
- 心跳/租约机制防死锁
- 并行分支设Join超时

挑战3：幻觉传播

- 上游错误被下游放大
- "看似一致"不等于"真实正确"

应对：

- 关键结论必须附证据链接
- 独立验证Agent二次核查
- 高风险输出强制人工签核

挑战4：安全与合规

- 数据泄露风险（跨Agent扩散）
- 提示注入导致越权执行
- 审计追责链不完整

应对：

- 最小权限 + 数据分级
- 工具白名单 + 参数校验
- 全链路审计日志

挑战5：组织协作

- 技术团队与业务团队语言不一致
- PoC好看，生产难落地

建议：

- 产品/风控/工程共同定义验收标准
- 先做"单流程标杆"，再平台化

治理框架（可落地）

层	要回答的问题
目标层	业务指标是否改善?
流程层	哪些步骤可自动化?
角色层	谁负责、谁审批、谁兜底?
技术层	如何观测、回滚、审计?
风险层	如何防泄露、防越权、防失控?

上线前检查清单（Go-Live）

- [ ] 成本上限与熔断策略
- [ ] 失败回退与人工接管流程
- [ ] 关键节点审计日志
- [ ] 指标仪表盘（质/效/成本）
- [ ] 安全评估与权限最小化

课堂复盘｜5分钟体验后的上线判断

请基于体验给出结论：

- 这类Agent产品适合你当前业务吗？
- 若上线，第一条治理红线是什么？
- 你会先从哪个低风险流程试点？

结论格式：通过 / 有条件通过 / 不通过（并说明理由）。

Part F｜课后项目案例

AI投研团队（从需求到交付）

案例背景

说明：本案例定义为课后项目（小组作业），不要求在课堂内实时完成。

目标：48小时内完成"AI芯片赛道投资备忘录"

约束：

- 数据来源多且噪声高
- 决策风险高
- 需要可追溯证据链

角色设计（示例）

- Supervisor：任务拆解与调度
- Research Agent：行业与竞品事实收集
- Finance Agent：估值模型与敏感性分析
- Risk Agent：政策/供应链风险
- Writer Agent：整合输出
- QA Agent：逻辑一致性审查

流程设计（混合模式）

1. Supervisor拆任务（Supervisor）
2. 三专家并行产出（Swarm并行）
3. Risk与Finance辩论关键假设（Debate）
4. Writer汇总并格式化（Pipeline）
5. QA签核后出报告

交付物定义

- `executive_summary.md`：1页高管摘要
- `evidence_pack.json`：证据与来源
- `risk_register.xlsx`：风险台账
- `decision_log.md`：关键决策记录

课堂演示脚本（可直接用）

```
Supervisor
"AI              "   5
                   Agent
```

评估维度（课堂评分）

维度	权重	说明
业务价值	30%	对决策是否有帮助
准确性	25%	证据是否可靠
可解释性	20%	过程是否可追踪
成本效率	15%	token/时间是否合理
风险治理	10%	是否有兜底机制

课后作业

题目：设计一个"多Agent市场进入分析系统"

要求：

1. 至少 4 个角色
2. 至少 2 种协作模式
3. 给出消息Schema
4. 说明为何选择对应框架（可含 LangGraph/Semantic Kernel/Dify/Coze 等）
5. 提交：架构图 + 关键Prompt + 一次运行日志

参考资源（必读）

官方指南 ⭐

- LangChain: Choosing the Right Multi-Agent Architecture - subagents/skills/handoffs/routers
- OpenAI: Multi-Agent Orchestration - Manager/Decentralized模式
- Anthropic: Multi-Agent Research System - 90.2%改进

框架

- Microsoft AutoGen
- CrewAI
- LangGraph 官方文档
- Semantic Kernel 官方文档

产品

课后延伸（选读）

- Agent评测与基准
- Tool Use安全机制
- 多Agent与MCP生态整合
- 智能体的组织设计（Org Design for AI）

一页总复盘

1. 多Agent价值来自"组织能力"而非"模型堆叠"
2. 四种模式是设计底盘
3. 通信与状态是工程成败关键
4. 框架选型要服务业务目标与治理要求
5. 落地需要成本、风险、可观测三位一体

Q&A

下一步：把你的业务流程画成"角色图 + 消息图"

建议会后立即做两件事： 1) 选一个真实业务流程做PoC； 2) 用今天的检查清单做一次上线评审。

谢谢

第9-10课时结束

下节课：MCP协议与工具生态（从Agent到可调用能力网络）